

Tugas Kecil 3 IF2211 Strategi Algoritma

Semester II tahun 2025/2026

Ice Sliding Puzzle Solver



Disusun oleh:

Muhammad Azzam Robbani

18223025

Leonard Arif Sutiono

18223120

PROGRAM STUDI SISTEM DAN TEKNOLOGI INFORMASI

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA

INSTITUT TEKNOLOGI BANDUNG

2025

Daftar Isi

Daftar Isi.....	2
1. Algoritma Pathfinding yang Digunakan.....	3
1.1 Representasi State dan Fungsi Slide.....	3
1.2 Uniform Cost Search (UCS).....	4
1.3 Greedy Best-First Search (GBFS).....	5
1.4 A* Search.....	5
1.5 Weighted A* (WA*).....	6
1.6 Fungsi Heuristik.....	6
2. Analisis Algoritma Pathfinding.....	8
2.1 Definisi $f(n)$ dan $g(n)$ pada Setiap Algoritma.....	8
2.2 Admissibility Heuristik pada A*.....	8
2.3 Perbandingan Teoritis Algoritma.....	9
3. Source Program.....	10
3.1 model.go.....	10
3.2 parser.go.....	11
3.3 pq.go.....	13
3.4 heuristic.go.....	13
3.5 solver.go.....	15
3.6 gui.go.....	17
3.7 main.go.....	33
4. Tangkapan Layar Input dan Output.....	35
4.1 Test Case 1: Pergerakan Dasar (Tanpa Angka).....	35
4.2 Test Case 2: Constraint Pengambilan Angka Berurutan.....	36
4.3 Test Case 3: Menghindari Halangan Lava.....	36
4.4 Test Case 4: Tidak Ada Solusi (Jalan Buntu).....	37
4.5 Test Case 5: Papan Skala Besar (Variasi Cost).....	37
5. Analisis Hasil Percobaan.....	39
6. Surat Pernyataan.....	41
LAMPIRAN.....	41

1. Algoritma Pathfinding yang Digunakan

Program Ice Sliding Puzzle Solver ini mengimplementasikan empat algoritma pathfinding: Uniform Cost Search (UCS) sebagai algoritma wajib, Greedy Best-First Search (GBFS) dan A* sebagai algoritma utama berbasis heuristik, serta Weighted A* (WA*) sebagai algoritma bonus. Seluruh algoritma bekerja pada ruang keadaan (state space) yang merepresentasikan posisi aktor dan digit berikutnya yang harus dikunjungi. Mekanisme pergerakan aktor mengikuti aturan ice sliding, yaitu aktor akan terus meluncur ke arah yang dipilih hingga menabrak dinding (X) atau batas papan, dan pergerakan yang melewati lava (L) atau digit yang tidak sesuai urutan dianggap tidak valid.

Karena sifat ice sliding, setiap "langkah" pada algoritma bukan satu perpindahan satu sel, melainkan satu aksi sliding penuh. Fungsi simulateSlide() menghitung hasil dari satu aksi tersebut, termasuk total cost traversal seluruh sel yang dilalui.

1.1 Representasi State dan Fungsi Slide

Setiap state direpresentasikan oleh pasangan (posisi aktor, digit berikutnya yang harus dikunjungi). State awal dibentuk dari posisi Z pada papan dengan NextDigit = 0. State tujuan tercapai ketika aktor berhenti tepat di posisi O dan semua digit telah dikunjungi sesuai urutan.

Fungsi inti yang menjadi dasar seluruh algoritma adalah simulateSlide(), yang mensimulasikan satu aksi pergerakan penuh dalam satu arah:

```
// simulateSlide: simulate one ice-sliding move in direction (dr, dc)
func simulateSlide(b *Board, s State, dr, dc int) (*State, int){
    r,c := s.Pos.R, s.Pos.C
    rr,cc := r,c
    totalCost := 0
    nextDigit := s.NextDigit
    moved := false
    for {
        nr := rr + dr
        nc := cc + dc
        if !b.InBounds(nr,nc) { return nil, 0 } // fell off board → invalid
        nxt := b.Tile(nr,nc)
        if nxt=='X' { break } // wall → stop here
        moved = true
        if nxt=='L' { return nil, 0 } // lava → invalid path
        if nxt>='0' && nxt<='9' {
            d := int(nxt - '0')
            if d > nextDigit { return nil, 0 } // wrong order → invalid
            if d == nextDigit { nextDigit++ } // correct digit collected
        }
        totalCost += b.TraversalCost(nr,nc)
        rr,cc = nr,nc
    }
    if !moved { return nil, 0 }
    ns := State{Pos:Position{rr,cc}, NextDigit:nextDigit}
```

```
    return &ns, totalCost
}
```

Fungsi ini menelusuri sel satu per satu ke arah yang ditentukan. Jika aktor keluar dari batas papan tanpa dinding, atau melewati lava, atau menginjak digit yang belum waktunya, maka aksi tersebut dinyatakan tidak valid dan fungsi mengembalikan nil. Biaya total pergerakan dihitung sebagai jumlah TraversalCost seluruh sel yang dilalui, tidak termasuk posisi awal namun termasuk posisi akhir.

1.2 Uniform Cost Search (UCS)

UCS adalah algoritma pencarian yang selalu mengekspansi node dengan akumulasi biaya terkecil dari state awal ($g(n)$). Algoritma ini tidak menggunakan heuristik sama sekali, sehingga eksplorasi dilakukan secara "buta" berdasarkan biaya aktual. UCS dijamin menemukan solusi dengan biaya minimum (optimal) selama seluruh biaya traversal bernilai non-negatif, yang terpenuhi dalam spesifikasi tugas ini.

Langkah-langkah UCS pada program ini:

1. Inisialisasi: State awal dimasukkan ke priority queue dengan prioritas $g = 0$. Peta $gbest[state] = 0$ menyimpan biaya terkecil yang pernah ditemukan ke state tersebut.
2. Ekspansi: Node dengan $g(n)$ terkecil di-dequeue. Jika ini adalah state tujuan, pencarian selesai.
3. Ekspansi tetangga: Untuk setiap arah (U/D/L/R), `simulateSlide()` dipanggil. Jika aksi valid, g baru dihitung sebagai $g(current) + stepCost$.
4. Pruning: Jika g baru tidak lebih baik dari $gbest[neighbor]$, node diabaikan. Jika lebih baik, $gbest$ diperbarui dan node dimasukkan ke frontier.
5. Rekonstruksi: Setelah goal ditemukan, jalur direkonstruksi menggunakan peta `parent[state]`.

1.3 Greedy Best-First Search (GBFS)

GBFS mengekspansi node berdasarkan nilai heuristik $h(n)$, yaitu estimasi jarak dari state saat ini ke tujuan, tanpa memperhitungkan biaya yang sudah dikeluarkan. Algoritma ini sangat cepat dalam banyak kasus, namun tidak dijamin menghasilkan solusi optimal karena sepenuhnya mengandalkan kualitas heuristik. Jika heuristik memandu ke arah yang benar, GBFS bisa menemukan solusi dengan sangat sedikit iterasi. Namun jika heuristik menyesatkan, GBFS dapat menemukan jalur yang mahal atau bahkan tidak menemukan solusi.

Pada program ini, GBFS menggunakan fungsi priority dengan formula $f(n) = h(n)$:

```
// priority: compute frontier priority depending on algorithm
func priority(algorithm string, g int, h float64, w float64) float64{
    switch algorithm {
    case "UCS":    return float64(g)           // f(n) = g(n)
    case "GBFS":   return h                     // f(n) = h(n)
    case "A*":     return float64(g) + h        // f(n) = g(n) + h(n)
    case "WASTAR": return float64(g) + w*h      // f(n) = g(n) + w·h(n)
    }
    panic("unknown algorithm")
}
```

1.4 A* Search

A* menggabungkan keunggulan UCS dan GBFS: ia menggunakan $f(n) = g(n) + h(n)$, di mana $g(n)$ adalah biaya aktual dari state awal ke state n , dan $h(n)$ adalah estimasi heuristik biaya dari n ke tujuan. Dengan demikian, A* mempertimbangkan baik biaya yang sudah dikeluarkan maupun perkiraan biaya yang masih harus dikeluarkan. Jika heuristik bersifat admissible (tidak pernah melebihi biaya aktual) dan consistent, A* dijamin menemukan solusi optimal.

Berikut adalah loop utama algoritma pencarian yang digunakan oleh UCS, GBFS, A*, maupun WA* — perbedaannya hanya pada perhitungan prioritas:

```
// Main search loop inside Solve()
for frontier.Len()>0 {
    it := frontier.PopItem()           // dequeue lowest-priority node
    current := it.State
    currentG := gbest[current]
    expanded++
    exploredOrder = append(exploredOrder, current)

    if isGoal(b, current){ goalState = &current; break }

    for _, dir := range DIRECTIONS {
        nxt, stepCost := simulateSlide(b, current, dir.DR, dir.DC)
        if nxt==nil { continue }
        nxtG := currentG + stepCost
        old, ok := gbest[*nxt]
        if ok && nxtG >= old { continue } // prune: not better than known
        gbest[*nxt] = nxtG
        // ... record parent, push to frontier
    }
}
```

Loop ini meng-dequeue node dengan prioritas terendah, memeriksa apakah state tujuan telah tercapai, lalu mengekskansi seluruh aksi yang valid. Mekanisme pruning gbest memastikan bahwa setiap state hanya diproses ulang jika ditemukan jalur yang lebih murah, sehingga efisiensi algoritma terjaga.

1.5 Weighted A* (WA*)

Weighted A* adalah varian A* yang memperbesar pengaruh heuristik dengan faktor bobot $w > 1$, sehingga formula prioritasnya menjadi $f(n) = g(n) + w \cdot h(n)$. Dengan $w = 1$, WA* identik dengan A*. Dengan $w > 1$, algoritma menjadi lebih "greedy" dan pada umumnya menemukan solusi lebih cepat dengan lebih sedikit iterasi, namun solusi yang dihasilkan hanya dijamin berada dalam faktor w dari optimal (disebut w -admissible). Pada implementasi ini, nilai default $w = 1.8$ digunakan.

WA* sangat berguna pada papan yang besar atau kompleks di mana A* membutuhkan terlalu banyak iterasi, sebagai trade-off antara kecepatan dan optimalitas.

1.6 Fungsi Heuristik

Program mengimplementasikan lima fungsi heuristik yang dapat dipilih pengguna. Setiap heuristik mengestimasi jarak dari state saat ini ke target berikutnya (digit yang harus dikunjungi, atau goal jika semua digit sudah terkunjungi):

```
// Five heuristic functions (heuristic.go)
func H1(b *Board, s State) float64 {           // Manhattan to next target
    t := targetForState(b, s)
    return float64(manhattan(s.Pos, t))
}
func H2(b *Board, s State) float64 {           // Manhattan + lookahead to goal
    t := targetForState(b, s)
    return float64(manhattan(s.Pos, t)) + 0.5*float64(manhattan(t, b.Goal))
}
func H3(b *Board, s State) float64 {           // Chebyshev distance
    t := targetForState(b, s)
    dr := abs(s.Pos.R - t.R); dc := abs(s.Pos.C - t.C)
    if dr>dc { return float64(dr) }; return float64(dc)
}
func H4(b *Board, s State) float64 {           // Euclidean distance
    t := targetForState(b, s)
    dr := float64(s.Pos.R-t.R); dc := float64(s.Pos.C-t.C)
    return math.Sqrt(dr*dr + dc*dc)
}
func H5(b *Board, s State) float64 {           // Manhattan + remaining digit
    penalty
    t := targetForState(b, s)
    base := manhattan(s.Pos, t)
    remaining := b.MaxDigit() - s.NextDigit + 1
    if remaining < 0 { remaining = 0 }
    return float64(base + 2*remaining)
}
```

1. **H1 (Manhattan):** Heuristik paling sederhana. Menghitung jarak Manhattan (selisih baris + selisih kolom) dari posisi aktor ke target berikutnya. Admissible karena jarak sliding selalu lebih besar atau sama dengan jarak Manhattan.

2. **H2 (Manhattan + Lookahead):** Menjumlahkan Manhattan ke target berikutnya dengan $0.5 \times$ Manhattan dari target ke goal. Memberikan pandangan lebih jauh, namun koefisien 0.5 dipilih agar tetap admissible.
3. **H3 (Chebyshev):** Menggunakan jarak Chebyshev (maksimum dari selisih baris dan selisih kolom). Cocok sebagai estimasi lower bound pada gerakan diagonal, namun pada ice sliding (hanya 4 arah) cenderung underestimate.
4. **H4 (Euclidean):** Jarak garis lurus antara posisi aktor dan target. Secara umum admissible karena jarak Euclidean tidak pernah melebihi jarak grid.
5. **H5 (Manhattan + Digit Penalty):** H1 ditambah penalti $2 \times$ jumlah digit yang belum dikunjungi. Heuristik ini lebih informed dan mendorong algoritma untuk menyelesaikan digit tersisa lebih cepat, namun perlu diverifikasi admissibility-nya (lihat Bagian 2).

2. Analisis Algoritma Pathfinding

2.1 Definisi $f(n)$ dan $g(n)$ pada Setiap Algoritma

Pada seluruh algoritma yang diimplementasikan, $g(n)$ didefinisikan secara konsisten sebagai total biaya traversal dari state awal ke state n , dihitung sebagai akumulasi biaya seluruh sel yang dilalui selama aksi-aksi sliding dari state awal. Biaya ini disimpan pada struktur $gbest[state]$ dan diperbarui setiap kali ditemukan jalur yang lebih murah ke suatu state.

Algoritma	$f(n)$	Keterangan
UCS	$f(n) = g(n)$	Hanya mempertimbangkan biaya aktual; tidak menggunakan heuristik.
GBFS	$f(n) = h(n)$	Hanya mempertimbangkan estimasi heuristik; mengabaikan biaya aktual.
A*	$f(n) = g(n) + h(n)$	Menggabungkan biaya aktual dan estimasi; optimal jika h admissible.
WA*	$f(n) = g(n) + w \cdot h(n)$	Varian A* berbobot; cepat namun hanya w -optimal ($w = 1.8$ pada impl. ini).

2.2 Admissibility Heuristik pada A*

Sebuah heuristik $h(n)$ disebut admissible jika untuk setiap state n berlaku $h(n) \leq h^*(n)$, di mana $h^*(n)$ adalah biaya aktual minimum dari n ke tujuan. Jika h admissible, A* dijamin menemukan solusi optimal. Berikut analisis admissibility untuk setiap heuristik yang diimplementasikan:

1. **H1 (Manhattan Distance):** Admissible. Pada ice sliding, aktor harus berpindah setidaknya sebanyak jarak Manhattan untuk mencapai target. Karena setiap sel yang dilalui memiliki biaya minimal 1 (berdasarkan spesifikasi cost non-negatif), dan aktor harus melewati minimal $|\Delta r| + |\Delta c|$ sel, maka $H1 \leq h^*(n)$. H1 merupakan heuristik yang consistent dan admissible.
2. **H2 (Manhattan + $0.5 \times \text{Lookahead}$):** Admissible, dengan catatan koefisien 0.5 dipilih secara konservatif. Suku pertama (Manhattan ke target berikutnya) tidak melebihi biaya aktual ke target tersebut. Suku kedua ($0.5 \times \text{Manhattan}$ dari target ke goal) menggunakan faktor reduksi sehingga penjumlahan masih tidak melebihi biaya aktual total. Admissibility bergantung pada distribusi cost pada papan; pada kasus umum dengan cost ≥ 1 per sel, H2 bersifat admissible.
3. **H3 (Chebyshev Distance):** Admissible. Chebyshev distance $\leq$ Manhattan distance, sehingga $H3 \leq H1 \leq h^*(n)$. Heuristik ini lebih lemah (less informed) dari H1, sehingga A* dengan H3 akan mengeksplorasi lebih banyak node dibandingkan H1.
4. **H4 (Euclidean Distance):** Admissible. Jarak Euclidean selalu lebih kecil atau sama dengan jarak Manhattan (karena jarak garis lurus tidak pernah melebihi jarak grid), sehingga $H4 \leq H1 \leq h^*(n)$. Heuristik ini juga less informed dari H1 pada grid.
5. **H5 (Manhattan + Digit Penalty):** Berpotensi tidak admissible pada beberapa kasus. Suku penalti $2 \times \text{remaining_digits}$ diasumsikan merepresentasikan biaya minimum untuk menyelesaikan digit tersisa, namun asumsi ini tidak selalu terpenuhi jika digit-digit tersebut berada berdekatan dan dapat diselesaikan dengan biaya sangat rendah. Pada praktiknya, H5 berfungsi sebagai heuristik informed yang agresif dan sangat efektif dalam mengurangi jumlah iterasi, meskipun optimalitas solusi tidak dijamin secara teoritis.

2.3 Perbandingan Teoritis Algoritma

Berikut perbandingan teoritis keempat algoritma pada permasalahan Ice Sliding Puzzle:

Aspek	UCS	GBFS	A*	WA*
Optimalitas	Optimal	Tidak dijamin	Optimal (h admissible)	w-Optimal
Kelengkapan	Lengkap	Tidak dijamin	Lengkap	Lengkap
Kompleksitas Waktu	$O(b^{(C^*/\epsilon)})$	$O(b^m)$	$O(b^d)$	Lebih kecil dari A*
Penggunaan Heuristik	Tidak ada	$h(n)$ saja	$g(n) + h(n)$	$g(n) + w \cdot h(n)$

Kecepatan (praktis)	Lambat	Cepat	Sedang	Cepat
Memori	Tinggi	Tinggi	Tinggi	Tinggi

Pada permasalahan Ice Sliding Puzzle yang memiliki cost traversal bervariasi per sel, UCS adalah yang paling cepat secara iterasi namun rentan menemukan jalur sub-optimal. A* menawarkan keseimbangan terbaik antara kecepatan dan optimalitas, terutama dengan heuristik yang baik seperti H1 atau H5. WA* berguna pada papan yang sangat besar di mana A* terlalu lambat, dengan mengorbankan sedikit optimalitas demi kecepatan yang signifikan. enjadi referensi optimalitas namun paling lambat karena tidak memiliki panduan heuristik. GBFS a

3. Source Program

Program ditulis dalam bahasa pemrograman Go dan terdiri dari enam file utama. Berikut adalah source code lengkap dari setiap file:

3.1 model.go

```
package main

type Position struct{
    R int
    C int
}

type State struct{
    Pos Position
    NextDigit int
}

type Board struct{
    Rows int
    Cols int
    Grid [][]rune
    Costs [][]int
    Start Position
    Goal Position
    DigitPositions map[int]Position
}

func (b *Board) MaxDigit() int{
    if len(b.DigitPositions)==0 { return -1 }
    m := -1
    for k := range b.DigitPositions{ if k>m { m=k } }
    return m
}
```

```

func (b *Board) InBounds(r,c int) bool{ return r>=0 && r<b.Rows && c>=0 && c<b.Cols
}

func (b *Board) Tile(r,c int) rune{ return b.Grid[r][c] }

func (b *Board) TraversalCost(r,c int) int{ return b.Costs[r][c] }

func RenderBoard(b *Board, actor Position, nextDigit int) []string{
    out := make([]string, 0, b.Rows)
    for r:=0; r<b.Rows; r++){
        row := make([]rune, b.Cols)
        for c:=0; c<b.Cols; c++){
            if r==actor.R && c==actor.C{ row[c]='Z'; continue }
            ch := b.Grid[r][c]
            if ch>='0' && ch<='9' {
                d := int(ch - '0')
                if d < nextDigit { row[c]='*'; continue }
            }
            if ch=='Z' { row[c]='*' } else { row[c]=ch }
        }
        out = append(out, string(row))
    }
    return out
}

```

3.2 parser.go

```

package main

import (
    "bufio"
    "fmt"
    "os"
    "strconv"
    "strings"
)

type ParseError string

func (e ParseError) Error() string { return string(e) }

func readNonEmptyLines(path string) ([]string, error){
    f, err := os.Open(path)
    if err!=nil { return nil, err }
    defer f.Close()
    return readNonEmptyLinesFromScanner(bufio.NewScanner(f))
}

func readNonEmptyLinesFromText(text string) ([]string, error){
    return readNonEmptyLinesFromScanner(bufio.NewScanner(strings.NewReader(text)))
}

func readNonEmptyLinesFromScanner(s *bufio.Scanner) ([]string, error){
    lines := []string{}
    for s.Scan(){
        t := strings.TrimRight(s.Text(), "\n\r")
        if strings.TrimSpace(t)==" " { continue }
        lines = append(lines, t)
    }
}

```

```

    }
    return lines, s.Err()
}

func LoadBoardFromText(text string) (*Board, error){
    lines, err := readNonEmptyLinesFromText(text)
    if err!=nil { return nil, err }
    return parseBoardLines(lines)
}

func LoadBoard(path string) (*Board, error){
    lines, err := readNonEmptyLines(path)
    if err!=nil { return nil, err }
    return parseBoardLines(lines)
}

func parseBoardLines(lines []string) (*Board, error){
    if len(lines) < 3 { return nil, ParseError("Input terlalu pendek.") }

    header := strings.Fields(lines[0])
    if len(header)!=2 { return nil, ParseError("Baris pertama harus berisi dua
bilangan: N M.") }
    n, err := strconv.Atoi(header[0])
    if err!=nil { return nil, ParseError("N dan M harus berupa bilangan bulat.") }
    m, err := strconv.Atoi(header[1])
    if err!=nil { return nil, ParseError("N dan M harus berupa bilangan bulat.") }
    if n<=0 || m<=0 { return nil, ParseError("Ukuran papan harus positif.") }

    expected := 1 + n + n
    if len(lines)!=expected { return nil, ParseError(fmt.Sprintf("Jumlah baris
tidak sesuai. Ditemukan %d, seharusnya %d.", len(lines), expected)) }

    gridLines := lines[1:1+n]
    costLines := lines[1+n:1+n+n]

    allowed := map[rune]bool{}
    for _, ch := range "XLOZ0123456789" { allowed[ch]=true }

    var startPos Position
    var goalPos Position
    gotStart := false
    gotGoal := false
    digitPositions := map[int]Position{}
    grid := make([][]rune, n)
    for r, line := range gridLines{
        if len(line) != m { return nil, ParseError(fmt.Sprintf("Panjang baris papan
ke-%d tidak sama dengan M=%d.", r+1, m)) }
        row := []rune(line)
        for c, ch := range row{
            if !allowed[ch] { return nil, ParseError(fmt.Sprintf("Karakter tidak
valid '%c' di (%d, %d).", ch, r, c)) }
            if ch=='Z' {
                if gotStart { return nil, ParseError("Aktor Z harus tepat satu.") }
                startPos = Position{r,c}; gotStart = true
            } else if ch=='O' {
                if gotGoal { return nil, ParseError("Tujuan O harus tepat satu.") }
                goalPos = Position{r,c}; gotGoal = true
            } else if ch>='0' && ch<='9' {
                d := int(ch - '0')
                if _,ok:=digitPositions[d]; ok { return nil,
ParseError(fmt.Sprintf("Digit %d muncul lebih dari sekali.", d)) }
                digitPositions[d] = Position{r,c}
            }
        }
    }

```

```

    }
    }
    grid[r] = row
    }
    if !gotStart { return nil, ParseError("Aktor Z tidak ditemukan.") }
    if !gotGoal { return nil, ParseError("Tujuan O tidak ditemukan.") }
    if len(digitPositions)>0{
        maxd := -1
        keys := map[int]bool{}
        for k := range digitPositions{ if k>maxd { maxd=k }; keys[k]=true }
        for i:=0;i<=maxd;i++){ if !keys[i]{ return nil, ParseError("Digit harus
berurutan mulai dari 0 tanpa celah.") } }
    }

    costs := make([][]int, n)
    for r, line := range costLines{
        parts := strings.Fields(line)
        if len(parts)!=m { return nil, ParseError(fmt.Sprintf("Jumlah cost pada baris
cost ke-%d tidak sama dengan M=%d.", r+1, m)) }
        row := make([]int, m)
        for i, p := range parts{
            v, err := strconv.Atoi(p)
            if err!=nil { return nil, ParseError(fmt.Sprintf("Cost tidak valid pada
baris cost ke-%d.", r+1)) }
            if v<0 { return nil, ParseError("Cost tidak boleh negatif.") }
            row[i]=v
        }
        costs[r]=row
    }

    return &Board{
        Rows:n, Cols:m, Grid:grid, Costs:costs, Start:startPos, Goal:goalPos,
        DigitPositions:digitPositions,
    }, nil
}

```

3.3 pq.go

```
package main

import "container/heap"

type PQItem struct{
    Priority float64
    Serial int
    State State
}

type PriorityQueue []PQItem

func (pq PriorityQueue) Len() int { return len(pq) }
func (pq PriorityQueue) Less(i,j int) bool{
    if pq[i].Priority==pq[j].Priority { return pq[i].Serial < pq[j].Serial }
    return pq[i].Priority < pq[j].Priority
}
func (pq PriorityQueue) Swap(i,j int){ pq[i],pq[j]=pq[j],pq[i] }
func (pq *PriorityQueue) Push(x interface{}){ *pq = append(*pq, x.(PQItem)) }
func (pq *PriorityQueue) Pop() interface{}{
    old := *pq
    n := len(old)
    it := old[n-1]
    *pq = old[:n-1]
    return it
}

func (pq *PriorityQueue) PushItem(item PQItem){ heap.Push(pq, item) }
func (pq *PriorityQueue) PopItem() PQItem{ return heap.Pop(pq).(PQItem) }
```

3.4 heuristic.go

```
package main

import "math"

func manhattan(a,b Position) int{ return abs(a.R-b.R)+abs(a.C-b.C) }
func abs(x int) int{ if x<0 { return -x }; return x }

func targetForState(b *Board, s State) Position{
    if s.NextDigit <= b.MaxDigit(){
        return b.DigitPositions[s.NextDigit]
    }
    return b.Goal
}

func H1(b *Board, s State) float64{
    t := targetForState(b,s)
    return float64(manhattan(s.Pos,t))
}

func H2(b *Board, s State) float64{
    t := targetForState(b,s)
    toTarget := manhattan(s.Pos,t)
```

```

        targetToGoal := manhattan(t,b.Goal)
        return float64(toTarget) + 0.5*float64(targetToGoal)
    }

    func H3(b *Board, s State) float64{
        t := targetForState(b,s)
        dr := abs(s.Pos.R - t.R)
        dc := abs(s.Pos.C - t.C)
        if dr>dc { return float64(dr) }
        return float64(dc)
    }

    func H4(b *Board, s State) float64{
        t := targetForState(b,s)
        dr := float64(s.Pos.R - t.R)
        dc := float64(s.Pos.C - t.C)
        return math.Sqrt(dr*dr + dc*dc)
    }

    func H5(b *Board, s State) float64{
        t := targetForState(b,s)
        base := manhattan(s.Pos,t)
        remaining := b.MaxDigit() - s.NextDigit + 1
        if remaining < 0 { remaining = 0 }
        return float64(base + 2*remaining)
    }

    type HeuristicFunc func(*Board, State) float64

    var Heuristics = map[string]HeuristicFunc{
        "H1": H1,
        "H2": H2,
        "H3": H3,
        "H4": H4,
        "H5": H5,
    }

```

3.5 solver.go

```

package main

import (
    "errors"
)

var DIRECTIONS = []struct{ Name string; DR, DC int }{
    {"U", -1, 0}, {"D", 1, 0}, {"L", 0, -1}, {"R", 0, 1},
}

type SolveResult struct{
    Found bool
    Moves string
    TotalCost int
    Expanded int
    ExploredOrder []State
    StatePath []State
}

```

```

func initialState(b *Board) State{ return State{Pos:b.Start, NextDigit:0} }

func isGoal(b *Board, s State) bool{ return s.Pos==b.Goal && s.NextDigit >
b.MaxDigit() }

func simulateSlide(b *Board, s State, dr, dc int) (*State, int){
    r,c := s.Pos.R, s.Pos.C
    rr,cc := r,c
    totalCost := 0
    nextDigit := s.NextDigit
    moved := false
    for {
        nr := rr + dr
        nc := cc + dc
        if !b.InBounds(nr,nc) { return nil, 0 }
        nxt := b.Tile(nr,nc)
        if nxt=='X' { break }
        moved = true
        if nxt=='L' { return nil, 0 }
        if nxt>='0' && nxt<='9' {
            d := int(nxt - '0')
            if d > nextDigit { return nil, 0 }
            if d == nextDigit { nextDigit++ }
        }
        totalCost += b.TraversalCost(nr,nc)
        rr,cc = nr,nc
    }
    if !moved { return nil, 0 }
    ns := State{Pos:Position{rr,cc}, NextDigit:nextDigit}
    return &ns, totalCost
}

func priority(algorithm string, g int, h float64, w float64) float64{
    switch algorithm {
        case "UCS": return float64(g)
        case "GBFS": return h
        case "A*": return float64(g) + h
        case "WASTAR": return float64(g) + w*h
    }
    panic("unknown algorithm")
}

func Solve(b *Board, algorithm, heuristicName string, weighted float64)
(SolveResult, error){
    if algorithm!="UCS" && algorithm!="GBFS" && algorithm!="A*" &&
algorithm!="WASTAR"{
        return SolveResult{}, errors.New("Algoritma harus UCS, GBFS, A*, atau
WASTAR.")
    }
    hfunc, ok := Heuristics[heuristicName]
    if !ok { return SolveResult{}, errors.New("Heuristik tidak dikenal:
"+heuristicName) }

    initial := initialState(b)
    frontier := &PriorityQueue{}
    serial := 0
    gbest := map[State]int{ initial:0 }
    parent := map[State]*State{ initial:nil }
    parentMove := map[State]string{}
    exploredOrder := []State{}

    firstH := hfunc(b, initial)

```

```

    frontier.PushItem(PQItem{Priority:priority(algorithm,0,firstH,weighted),
Serial:serial, State:initial})
    serial++

    expanded := 0
    var goalState *State

    for frontier.Len()>0 {
        it := frontier.PopItem()
        current := it.State
        currentG := gbest[current]
        expanded++
        exploredOrder = append(exploredOrder, current)
        if isGoal(b, current){ goalState = &current; break }
        for _, dir := range DIRECTIONS{
            nxt, stepCost := simulateSlide(b, current, dir.DR, dir.DC)
            if nxt==nil { continue }
            nxtG := currentG + stepCost
            old, ok := gbest[*nxt]
            if ok && nxtG >= old { continue }
            gbest[*nxt] = nxtG
            // store copy
            ncopy := *nxt
            parent[ncopy] = &current
            parentMove[ncopy] = dir.Name
            hval := hfunc(b, ncopy)
            pri := priority(algorithm, nxtG, hval, weighted)
            frontier.PushItem(PQItem{Priority:pri, Serial:serial, State:ncopy})
            serial++
        }
    }

    if goalState==nil{
        return SolveResult{Found:false, Moves:"", TotalCost:0, Expanded:expanded,
ExploredOrder:exploredOrder, StatePath:[]State{}}, nil
    }

    // reconstruct path
    movesRev := []string{}
    pathRev := []State{}
    cur := *goalState
    for {
        pathRev = append(pathRev, cur)
        prev := parent[cur]
        if prev!=nil {
            movesRev = append(movesRev, parentMove[cur])
        }
        if prev==nil { break }
        cur = *prev
    }
    // reverse
    path := make([]State, len(pathRev))
    for i:=0;i<len(pathRev);i++{ path[i]=pathRev[len(pathRev)-1-i] }
    moves := ""
    for i:=len(movesRev)-1;i>=0;i--{ moves += movesRev[i] }

    return SolveResult{Found:true, Moves:moves, TotalCost: gbest[*goalState],
Expanded:expanded, ExploredOrder:exploredOrder, StatePath:path}, nil
}

```


3.6 gui.go

```
package main

import (
    "encoding/json"
    "fmt"
    "net"
    "net/http"
    "os/exec"
    "runtime"
    "strconv"
    "strings"
    "sync"
    "time"
)

type guiSession struct {
    Board      *Board
    Result      SolveResult
    ElapsedMS   float64
    Algorithm   string
    Heuristic   string
    Weighted    float64
    InputLabel  string
    Frames      [][]string
    FrameMoves []string
    Error       string
}

type guiServer struct {
    mu      sync.RWMutex
    session *guiSession
}

func runGUI() {
    server := &guiServer{}
    mux := http.NewServeMux()
    mux.HandleFunc("/", server.handleIndex)
    mux.HandleFunc("/solve", server.handleSolve)
    mux.HandleFunc("/download", server.handleDownload)

    listener, err := net.Listen("tcp", "127.0.0.1:0")
    if err != nil {
        fmt.Println("Gagal memulai server GUI:", err)
        return
    }

    baseURL := "http://" + listener.Addr().String()
    go func() {
        _ = http.Serve(listener, mux)
    }()

    if err := openBrowser(baseURL); err != nil {
        fmt.Println("Server GUI aktif di", baseURL)
        fmt.Println("Gagal membuka browser otomatis:", err)
    } else {
        fmt.Println("GUI aktif di", baseURL)
    }
}
```

```

        select {}
    }

    func (s *guiServer) handleIndex(w http.ResponseWriter, r *http.Request) {
        w.Header().Set("Content-Type", "text/html; charset=utf-8")
        _, _ = w.Write([]byte(guiHTML))
    }

    func (s *guiServer) handleSolve(w http.ResponseWriter, r *http.Request) {
        if r.Method != http.MethodPost {
            http.Error(w, "Method not allowed", http.StatusMethodNotAllowed)
            return
        }
        if err := r.ParseForm(); err != nil {
            s.writeJSONError(w, http.StatusBadRequest, "Form tidak valid: "+err.Error())
            return
        }

        boardText := strings.TrimSpace(r.FormValue("boardText"))
        if boardText == "" {
            s.writeJSONError(w, http.StatusBadRequest, "Masukkan file papan terlebih dahulu.")
            return
        }

        board, err := LoadBoardFromText(boardText)
        if err != nil {
            s.writeJSONError(w, http.StatusBadRequest, err.Error())
            return
        }

        algorithm := strings.ToUpper(strings.TrimSpace(r.FormValue("algorithm")))
        if algorithm == "" {
            algorithm = "UCS"
        }
        heuristic := strings.ToUpper(strings.TrimSpace(r.FormValue("heuristic")))
        if heuristic == "" || heuristic == "-" {
            heuristic = "H1"
        }
        weighted := 1.8
        if rawWeighted := strings.TrimSpace(r.FormValue("weighted")); rawWeighted != "" {
            if parsed, parseErr := strconv.ParseFloat(rawWeighted, 64); parseErr == nil && parsed > 0 {
                weighted = parsed
            }
        }

        start := time.Now()
        result, solveErr := Solve(board, algorithm, heuristic, weighted)
        elapsed := float64(time.Since(start).Milliseconds())
        if solveErr != nil {
            s.writeJSONError(w, http.StatusBadRequest, solveErr.Error())
            return
        }

        frames := make([][]string, 0, len(result.StatePath))
        labels := make([]string, 0, len(result.StatePath))
        if len(result.StatePath) == 0 {
            frames = append(frames, RenderBoard(board, board.Start, 0))
            labels = append(labels, "Initial")
        }
    }

```

```

    } else {
        for i, state := range result.StatePath {
            frames = append(frames, RenderBoard(board, state.Pos,
state.NextDigit))
            if i == 0 {
                labels = append(labels, "Initial")
            } else if i-1 < len(result.Moves) {
                labels = append(labels, fmt.Sprintf("Step %d: %c", i,
result.Moves[i-1]))
            } else {
                labels = append(labels, fmt.Sprintf("Step %d", i))
            }
        }
    }

    session := &guiSession{
        Board:      board,
        Result:      result,
        ElapsedMS:   elapsed,
        Algorithm:   algorithm,
        Heuristic:   heuristic,
        Weighted:    weighted,
        InputLabel:  "uploaded board",
        Frames:      frames,
        FrameMoves: labels,
    }

    s.mu.Lock()
    s.session = session
    s.mu.Unlock()

    response := map[string]any{
        "ok":      true,
        "found":    result.Found,
        "moves":    result.Moves,
        "totalCost": result.TotalCost,
        "expanded": result.Expanded,
        "elapsedMS": elapsed,
        "algorithm": algorithm,
        "heuristic": heuristic,
        "weighted":  weighted,
        "frames":    frames,
        "frameMoves": labels,
    }

    w.Header().Set("Content-Type", "application/json; charset=utf-8")
    _ = json.NewEncoder(w).Encode(response)
}

func (s *guiServer) handleDownload(w http.ResponseWriter, r *http.Request) {
    s.mu.RLock()
    session := s.session
    s.mu.RUnlock()
    if session == nil {
        http.Error(w, "Belum ada hasil solusi.", http.StatusNotFound)
        return
    }

    filename := "solusi.txt"
    w.Header().Set("Content-Type", "text/plain; charset=utf-8")
    w.Header().Set("Content-Disposition", "attachment; filename="+filename)
    if session.Result.Found {
        _, _ = fmt.Fprintf(w, "Algoritma: %s\n", session.Algorithm)
    }
}

```

```

_, _ = fmt.Fprintf(w, "Heuristik: %s\n", session.Heuristic)
_, _ = fmt.Fprintf(w, "Weighted: %.3f\n", session.Weighted)
_, _ = fmt.Fprintf(w, "Solusi: %s\n", session.Result.Moves)
_, _ = fmt.Fprintf(w, "Cost: %d\n", session.Result.TotalCost)
_, _ = fmt.Fprintf(w, "Iterasi: %d\n", session.Result.Expanded)
_, _ = fmt.Fprintf(w, "Waktu(ms): %.3f\n\n", session.ElapsedMS)
for i, st := range session.Result.StatePath {
    if i == 0 {
        _, _ = fmt.Fprintln(w, "Initial")
    } else {
        _, _ = fmt.Fprintf(w, "Step %d: %c\n", i,
session.Result.Moves[i-1])
    }
    for _, line := range RenderBoard(session.Board, st.Pos,
st.NextDigit) {
        _, _ = fmt.Fprintln(w, line)
    }
    _, _ = fmt.Fprintln(w)
}
return
}
_, _ = fmt.Fprintln(w, "Tidak ada solusi valid.")
}

func (s *guiServer) writeJSONError(w http.ResponseWriter, status int, message
string) {
    w.Header().Set("Content-Type", "application/json; charset=utf-8")
    w.WriteHeader(status)
    _ = json.NewEncoder(w).Encode(map[string]any{
        "ok": false,
        "error": message,
    })
}

func openBrowser(rawURL string) error {
    escaped := rawURL
    if runtime.GOOS == "windows" {
        return exec.Command("rundll32", "url.dll,FileProtocolHandler",
escaped).Start()
    }
    if runtime.GOOS == "darwin" {
        return exec.Command("open", escaped).Start()
    }
    return exec.Command("xdg-open", escaped).Start()
}

var guiHTML = `<!doctype html>
<html lang="en">
<head>
    <meta charset="utf-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1" />
    <title>Ice Sliding Puzzle Solver</title>
    <style>
        :root {
            --bg: #07111f;
            --bg2: #0c1b2f;
            --card: rgba(9, 19, 35, 0.86);
            --card2: rgba(16, 31, 54, 0.9);
            --text: #eef4ff;
            --muted: #9fb3cf;
            --accent: #71d7ff;
            --accent2: #8a7dff;

```

```

        --ok: #56e39f;
        --warn: #ffbf69;
        --danger: #ff6b6b;
        --line: rgba(255,255,255,0.08);
        --shadow: 0 28px 80px rgba(0,0,0,0.35);
        --radius: 22px;
    }
    * { box-sizing: border-box; }
    body {
        margin: 0;
        min-height: 100vh;
        font-family: "Segoe UI", system-ui, sans-serif;
        color: var(--text);
        background:
            radial-gradient(circle at top left,
rgb(113,215,255,0.16), transparent 28%),
            radial-gradient(circle at top right,
rgb(138,125,255,0.18), transparent 25%),
            linear-gradient(160deg, var(--bg), var(--bg2));
    }
    .shell {
        width: min(1300px, calc(100vw - 32px));
        margin: 0 auto;
        padding: 24px 0 40px;
    }
    .hero {
        display: grid;
        gap: 18px;
        grid-template-columns: 1.4fr 0.9fr;
        align-items: end;
        margin-bottom: 18px;
    }
    .title {
        padding: 28px;
        border: 1px solid var(--line);
        border-radius: var(--radius);
        background: linear-gradient(135deg, rgba(15,30,54,0.92),
rgb(10,18,33,0.88));
        box-shadow: var(--shadow);
    }
    .title h1 {
        margin: 0 0 10px;
        font-size: clamp(30px, 4vw, 54px);
        line-height: 0.96;
        letter-spacing: -0.05em;
    }
    .title p {
        margin: 0;
        color: var(--muted);
        max-width: 68ch;
    }
    .pillbar {
        display: flex;
        gap: 10px;
        flex-wrap: wrap;
        justify-content: flex-end;
    }
    .pill {
        border: 1px solid var(--line);
        background: rgba(255,255,255,0.05);
        color: var(--text);
        padding: 10px 14px;
    }

```

```

        border-radius: 999px;
        font-size: 13px;
        backdrop-filter: blur(12px);
    }
    .grid {
        display: grid;
        grid-template-columns: 420px 1fr;
        gap: 18px;
    }
    .panel {
        border: 1px solid var(--line);
        background: var(--card);
        border-radius: var(--radius);
        box-shadow: var(--shadow);
        overflow: hidden;
    }
    .panel .head {
        padding: 18px 20px 14px;
        border-bottom: 1px solid var(--line);
        background: rgba(255,255,255,0.02);
    }
    .panel .head h2,
    .panel .head h3 {
        margin: 0;
        font-size: 16px;
        letter-spacing: 0.02em;
    }
    .panel .head .sub {
        margin-top: 6px;
        color: var(--muted);
        font-size: 13px;
    }
    .panel .body { padding: 18px 20px 20px; }
    .field {
        display: grid;
        gap: 8px;
        margin-bottom: 14px;
    }
    label { font-size: 13px; color: var(--muted); }
    input[type="text"], input[type="number"], select, textarea {
        width: 100%;
        border: 1px solid rgba(255,255,255,0.12);
        background: rgba(3, 10, 22, 0.75);
        color: var(--text);
        border-radius: 14px;
        padding: 12px 14px;
        outline: none;
        transition: border-color .18s ease, transform .18s ease,
box-shadow .18s ease;
    }
    textarea {
        min-height: 260px;
        resize: vertical;
        font-family: Consolas, "Cascadia Mono", monospace;
        line-height: 1.35;
        white-space: pre;
    }
    input:focus, select:focus, textarea:focus {
        border-color: rgba(113,215,255,0.7);
        box-shadow: 0 0 0 3px rgba(113,215,255,0.15);
        transform: translateY(-1px);
    }

```

```

        .actions {
            display: flex;
            gap: 10px;
            flex-wrap: wrap;
            margin-top: 8px;
        }
        button, .linkbtn {
            border: 0;
            background: linear-gradient(135deg, var(--accent),
var(--accent2));
            color: #06111e;
            font-weight: 700;
            padding: 12px 16px;
            border-radius: 14px;
            cursor: pointer;
            text-decoration: none;
            display: inline-flex;
            align-items: center;
            justify-content: center;
            transition: transform .18s ease, filter .18s ease;
        }
        button.secondary, .linkbtn.secondary {
            background: rgba(255,255,255,0.08);
            color: var(--text);
            border: 1px solid var(--line);
        }
        button:hover, .linkbtn:hover { transform: translateY(-1px); filter:
brightness(1.05); }
        button:disabled { opacity: 0.55; cursor: not-allowed; transform: none;
    }

    .status {
        display: grid;
        grid-template-columns: repeat(4, minmax(0, 1fr));
        gap: 12px;
        margin-bottom: 14px;
    }

    .stat {
        padding: 16px;
        border-radius: 18px;
        background: rgba(255,255,255,0.04);
        border: 1px solid var(--line);
    }

    .stat.k { color: var(--muted); font-size: 12px; text-transform:
uppercase; letter-spacing: .08em; }
    .stat.v { margin-top: 8px; font-size: 22px; font-weight: 700; }
    .stat.v.ok { color: var(--ok); }
    .stat.v.warn { color: var(--warn); }
    .workspace {
        display: grid;
        grid-template-columns: minmax(0, 1fr) 280px;
        gap: 16px;
        align-items: start;
    }

    .viewer {
        padding: 16px;
        border-radius: 18px;
        background: rgba(4, 12, 23, 0.92);
        border: 1px solid var(--line);
        min-height: 320px;
        overflow: hidden;
    }

    .board-shell {

```

```

        display: grid;
        gap: 12px;
    }
    .board-meta {
        display: flex;
        justify-content: space-between;
        align-items: center;
        gap: 12px;
        color: var(--muted);
        font-size: 13px;
        margin-bottom: 8px;
    }
    .board-grid {
        display: grid;
        gap: 4px;
        padding: 12px;
        border-radius: 18px;
        background:
            radial-gradient(circle at top, rgba(113,215,255,0.12),
transparent 28%),
            linear-gradient(180deg, rgba(7, 18, 35, 0.98), rgba(12,
26, 46, 0.99));
        border: 1px solid rgba(255,255,255,0.07);
        box-shadow: inset 0 1px 0 rgba(255,255,255,0.03);
        position: relative;
        transform-origin: center;
        transition: transform .2s ease, filter .2s ease;
    }
    .board-grid.is-changing {
        transform: scale(0.992);
        filter: brightness(1.04);
    }
    .tile {
        position: relative;
        width: 42px;
        height: 42px;
        border-radius: 10px;
        display: flex;
        align-items: center;
        justify-content: center;
        font-weight: 800;
        font-size: 17px;
        line-height: 1;
        letter-spacing: 0.04em;
        user-select: none;
        transition: transform .22s cubic-bezier(.2,.9,.2,1),
background-color .22s ease, box-shadow .22s ease, color .22s ease, opacity .22s
ease;
    }
    .tile::after {
        content: '';
        position: absolute;
        inset: 0;
        border-radius: inherit;
        border: 1px solid rgba(255,255,255,0.03);
        pointer-events: none;
    }
    .tile.wall { background: linear-gradient(180deg, #1a2331, #0f1723);
color: #60738d; }
    .tile.empty {
        background: linear-gradient(180deg, rgba(214,221,232,0.42),
rgba(176,186,202,0.28));
    }

```



```

        box-shadow: inset 0 1px 0 rgba(255,255,255,0.14), inset 0 -1px 0
rgba(0,0,0,0.18);
        color: transparent;
    }
    .tile.goal { background: linear-gradient(180deg, rgba(48,190,137,0.95),
rgba(28,139,98,0.95)); color: #03140c; box-shadow: 0 0 0 1px rgba(86,227,159,0.18),
0 8px 18px rgba(86,227,159,0.18); }
    .tile.block { background: linear-gradient(180deg,
rgba(173,100,255,0.9), rgba(103,61,194,0.9)); color: #fff; }
    .tile.digit { background: linear-gradient(180deg,
rgba(255,191,105,0.95), rgba(214,136,36,0.95)); color: #2a1400; box-shadow: 0 0 0
1px rgba(255,191,105,0.22), 0 10px 22px rgba(255,191,105,0.14); }
    .tile.start { background: radial-gradient(circle at 30% 30%, #7ef0ff,
#1293d0); color: #02121d; box-shadow: 0 0 0 1px rgba(113,215,255,0.22), 0 10px 24px
rgba(19,147,208,0.25); transform: scale(1.03); }
    .tile.hazard { background: linear-gradient(180deg,
rgba(255,110,110,0.92), rgba(183,57,57,0.92)); color: #fff3f3; }
    .tile.lit {
        animation: pulseGlow .7s ease-out;
    }
    .tile.moved {
        transform: translateY(-2px) scale(1.06);
    }
    .board-legend {
        display: flex;
        gap: 10px;
        flex-wrap: wrap;
        font-size: 12px;
        color: var(--muted);
    }
    .legend-item {
        display: inline-flex;
        align-items: center;
        gap: 8px;
        padding: 8px 10px;
        border-radius: 999px;
        background: rgba(255,255,255,0.04);
        border: 1px solid rgba(255,255,255,0.06);
    }
    .legend-swatch {
        width: 10px;
        height: 10px;
        border-radius: 999px;
    }
    @keyframes pulseGlow {
        0% { box-shadow: 0 0 0 0 rgba(113,215,255,0.0); }
        40% { box-shadow: 0 0 0 8px rgba(113,215,255,0.18); }
        100% { box-shadow: 0 0 0 0 rgba(113,215,255,0.0); }
    }
    .play {
        display: grid;
        gap: 12px;
        padding: 16px;
        border-radius: 18px;
        background: rgba(255,255,255,0.04);
        border: 1px solid var(--line);
    }
    .play .row {
        display: flex;
        gap: 10px;
        flex-wrap: wrap;
    }

```

```

.slider {
  width: 100%;
}
.footer-note {
  color: var(--muted);
  font-size: 12px;
  margin-top: 12px;
}
.error {
  color: #ffd2d2;
  background: rgba(255, 107, 107, 0.12);
  border: 1px solid rgba(255, 107, 107, 0.25);
  padding: 12px 14px;
  border-radius: 14px;
  margin-top: 12px;
  display: none;
}
.muted { color: var(--muted); }
.hidden { display: none !important; }
@media (max-width: 1100px) {
  .grid, .hero, .workspace { grid-template-columns: 1fr; }
  .pillbar { justify-content: flex-start; }
  .status { grid-template-columns: repeat(2, minmax(0, 1fr)); }
}
@media (max-width: 640px) {
  .shell { width: calc(100vw - 20px); }
  .title, .panel .body, .panel .head { padding-left: 16px;
padding-right: 16px; }
  .status { grid-template-columns: 1fr; }
  pre { font-size: 14px; }
}
</style>
</head>
<body>
  <div class="shell">
    <div class="hero">
      <div class="title">
        <h1>Ice Sliding Puzzle Solver</h1>
        <p>Upload a puzzle file, run UCS / GBFS / A* / WASTAR,
then inspect the solved path with playback, statistics, and a downloadable
report.</p>
      </div>
    </div>

    <div class="grid">
      <div class="panel">
        <div class="head">
          <h2>Input</h2>
          <div class="sub">Choose a file, or paste the raw
board text below.</div>
        </div>
        <div class="body">
          <form id="solveForm">
            <div class="field">
              <label for="fileInput">Input
file</label>
              <input id="fileInput" type="file"
accept=".txt,text/plain" />
            </div>
            <div class="field">
              <label for="boardText">Board
text</label>

```

```

name="boardText" placeholder="Paste puzzle text here..."></textarea>
</div>
<div class="field">
  <label
for="algorithm">Algorithm</label>
  <select id="algorithm"
name="algorithm">
    <option
value="UCS">UCS</option>
    <option
value="GBFS">GBFS</option>
    <option value="A*">A*</option>
    <option
value="WASTAR">WASTAR</option>
  </select>
</div>
<div class="field">
  <label
for="heuristic">Heuristic</label>
  <select id="heuristic"
name="heuristic">
    <option value="-">-</option>
    <option value="H1">H1</option>
    <option value="H2">H2</option>
    <option value="H3">H3</option>
    <option value="H4">H4</option>
    <option value="H5">H5</option>
  </select>
</div>
<div class="field">
  <label for="weighted">Weighted
factor</label>
  <input id="weighted" name="weighted"
type="number" min="1" step="0.1" value="1.8" />
</div>
<div class="actions">
  <button id="solveBtn"
type="button">Run solver</button>
  <a class="linkbtn secondary"
id="downloadBtn" href="/download" target="_blank">Download result</a>
</div>
</form>
<div id="errorBox" class="error"></div>
<div class="footer-note">Tip: selecting a file
auto-fills the text box so you can edit before solving.</div>
</div>

<div class="panel">
  <div class="head">
    <h2>Result</h2>
    <div class="sub">Solve output and step
    playback.</div>
  </div>
  <div class="body">
    <div class="status">
      <div class="stat"><div
class="k">Found</div><div id="statFound" class="v warn">-</div></div>
      <div class="stat"><div
class="k">Cost</div><div id="statCost" class="v">-</div></div>
      <div class="stat"><div

```

```

class="k">Expanded</div><div id="statExpanded" class="v">-</div></div>
      <div class="stat"><div
class="k">Elapsed</div><div id="statElapsed" class="v">-</div></div>
      </div>
      <div class="workspace">
        <div class="viewer">
          <div class="board-shell">
            <div class="board-meta">
              <div
id="boardTitle">Load a puzzle to see the board here.</div>
              <div id="boardCoords"
class="muted">0 x 0</div>
              </div>
            <div id="boardGrid"
class="board-grid"></div>
            <div class="board-legend">
              <span
class="legend-item"><span class="legend-swatch"
style="background:#1293d0"></span>Actor</span>
              <span
class="legend-item"><span class="legend-swatch"
style="background:#30be89"></span>Goal</span>
              <span
class="legend-item"><span class="legend-swatch"
style="background:#d68824"></span>Digit</span>
              <span
class="legend-item"><span class="legend-swatch"
style="background:#b73939"></span>Hazard</span>
              </div>
            </div>
            <div class="play">
              <div><strong
id="stepLabel">Playback</strong></div>
              <input id="frameSlider" class="slider"
type="range" min="0" max="0" value="0" disabled />
              <div class="row">
                <button id="prevBtn"
class="secondary" type="button" disabled>Prev</button>
                <button id="playBtn"
class="secondary" type="button" disabled>Play</button>
                <button id="nextBtn"
class="secondary" type="button" disabled>Next</button>
              </div>
              <div class="muted"
id="frameCounter">Step 0 of 0</div>
              <div class="footer-note">Use the
slider to inspect the reconstructed path one frame at a time.</div>
            </div>
          </div>
        </div>
      </div>
    </div>
  </div>
  <script>
    const fileInput = document.getElementById('fileInput');
    const boardText = document.getElementById('boardText');
    const solveForm = document.getElementById('solveForm');
    const errorBox = document.getElementById('errorBox');
    const solveBtn = document.getElementById('solveBtn');
    const statFound = document.getElementById('statFound');
  </script>

```

```

const statCost = document.getElementById('statCost');
const statExpanded = document.getElementById('statExpanded');
const statElapsed = document.getElementById('statElapsed');
const boardGrid = document.getElementById('boardGrid');
const boardTitle = document.getElementById('boardTitle');
const boardCoords = document.getElementById('boardCoords');
const stepLabel = document.getElementById('stepLabel');
const frameSlider = document.getElementById('frameSlider');
const frameCounter = document.getElementById('frameCounter');
const prevBtn = document.getElementById('prevBtn');
const playBtn = document.getElementById('playBtn');
const nextBtn = document.getElementById('nextBtn');
const downloadBtn = document.getElementById('downloadBtn');
const algorithm = document.getElementById('algorithm');
const heuristic = document.getElementById('heuristic');

let frames = [];
let labels = [];
let frameSize = { rows: 0, cols: 0 };
let playing = false;
let playTimer = null;
let cellNodes = [];
let previousActorIndex = -1;

function parseFrame(frame) {
    return frame.map((row) => row.split(''));
}

function tileClass(ch) {
    if (ch === 'X') return 'tile wall';
    if (ch === 'O') return 'tile goal';
    if (ch === 'L') return 'tile hazard';
    if (ch === 'Z') return 'tile start';
    if (ch >= '0' && ch <= '9') return 'tile digit';
    return 'tile empty';
}

function tileLabel(ch) {
    if (ch === 'X' || ch === '*') return '';
    if (ch === 'Z') return 'Z';
    return ch;
}

function findActorIndex(grid) {
    for (let r = 0; r < grid.length; r++) {
        for (let c = 0; c < grid[r].length; c++) {
            if (grid[r][c] === 'Z') return r * grid[r].length +
c;
        }
    }
    return -1;
}

function ensureBoard(grid) {
    if (!grid.length) return;
    if (cellNodes.length && frameSize.rows === grid.length &&
frameSize.cols === grid[0].length) return;
    frameSize = { rows: grid.length, cols: grid[0].length };
    boardGrid.innerHTML = '';
    boardGrid.style.gridTemplateColumns = 'repeat(' + frameSize.cols
+ ', minmax(42px, 1fr))';
    cellNodes = [];

```

```

        for (let r = 0; r < frameSize.rows; r++) {
            for (let c = 0; c < frameSize.cols; c++) {
                const cell = document.createElement('div');
                cell.className = 'tile empty';
                cell.dataset.row = String(r);
                cell.dataset.col = String(c);
                const span = document.createElement('span');
                span.className = 'tile-text';
                cell.appendChild(span);
                boardGrid.appendChild(cell);
                cellNodes.push(cell);
            }
        }
    }

    function animateBoardSwap(nextActorIndex) {
        boardGrid.classList.remove('is-changing');
        void boardGrid.offsetWidth;
        boardGrid.classList.add('is-changing');
        if (previousActorIndex >= 0 && previousActorIndex <
cellNodes.length) {
            cellNodes[previousActorIndex].classList.remove('moved');
            void cellNodes[previousActorIndex].offsetWidth;
            cellNodes[previousActorIndex].classList.add('moved');
        }
        if (nextActorIndex >= 0 && nextActorIndex < cellNodes.length) {
            cellNodes[nextActorIndex].classList.remove('moved');
            void cellNodes[nextActorIndex].offsetWidth;
            cellNodes[nextActorIndex].classList.add('moved');
        }
        previousActorIndex = nextActorIndex;
    }

    function renderBoard(frameIndex) {
        if (!frames.length) {
            boardTitle.textContent = 'No frames available.';
            boardCoords.textContent = '0 x 0';
            boardGrid.innerHTML = '';
            cellNodes = [];
            previousActorIndex = -1;
            return;
        }
        const safeIndex = Math.max(0, Math.min(frameIndex, frames.length
- 1));

        const grid = parseFrame(frames[safeIndex] || []);
        ensureBoard(grid);
        boardTitle.textContent = labels[safeIndex] || ('Step ' +
safeIndex);

        boardCoords.textContent = frameSize.rows + ' x ' +
frameSize.cols;

        let actorIndex = -1;
        for (let r = 0; r < frameSize.rows; r++) {
            for (let c = 0; c < frameSize.cols; c++) {
                const idx = r * frameSize.cols + c;
                const cell = cellNodes[idx];
                const ch = grid[r][c];
                cell.className = tileClass(ch);
                const text = tileLabel(ch);
                cell.querySelector('.tile-text').textContent =
text;

                if (ch === 'Z') actorIndex = idx;
            }
        }
    }

```

```

        }
        animateBoardSwap(actorIndex);
    }

    function showError(message) {
        errorBox.textContent = message;
        errorBox.style.display = 'block';
    }

    function clearError() {
        errorBox.textContent = '';
        errorBox.style.display = 'none';
    }

    function renderFrame(index) {
        if (!frames.length) {
            stepLabel.textContent = 'Playback';
            frameCounter.textContent = 'Step 0 of 0';
            renderBoard(-1);
            return;
        }
        const safeIndex = Math.max(0, Math.min(index, frames.length -
1));
        renderBoard(safeIndex);
        stepLabel.textContent = labels[safeIndex] || ('Step ' +
safeIndex);
        frameSlider.value = String(safeIndex);
        frameCounter.textContent = 'Step ' + (safeIndex + 1) + ' of ' +
frames.length;
    }

    function setPlaying(nextPlaying) {
        playing = nextPlaying;
        playBtn.textContent = playing ? 'Pause' : 'Play';
        if (playTimer) {
            clearInterval(playTimer);
            playTimer = null;
        }
        if (playing) {
            playTimer = setInterval(() => {
                const current = Number(frameSlider.value || 0);
                if (current >= frames.length - 1) {
                    setPlaying(false);
                    return;
                }
                renderFrame(current + 1);
            }, 700);
        }
    }

    fileInput.addEventListener('change', async () => {
        const file = fileInput.files && fileInput.files[0];
        if (!file) return;
        boardText.value = await file.text();
    });

    frameSlider.addEventListener('input', () =>
renderFrame(Number(frameSlider.value || 0)));
    prevBtn.addEventListener('click', () =>
renderFrame(Number(frameSlider.value || 0) - 1));
    nextBtn.addEventListener('click', () =>
renderFrame(Number(frameSlider.value || 0) + 1));

```

```

        playBtn.addEventListener('click', () => setPlaying(!playing));

        algorithm.addEventListener('change', () => {
            const active = algorithm.value === 'GBFS' || algorithm.value ===
'A*' || algorithm.value === 'WASTAR';
            if (algorithm.value === 'UCS') {
                heuristic.value = '-';
            } else if (heuristic.value === '-') {
                heuristic.value = 'H1';
            }
            heuristic.disabled = !active;
            document.getElementById('weighted').disabled = algorithm.value
!= 'WASTAR';
        });
        algorithm.dispatchEvent(new Event('change'));

        async function runSolver() {
            clearError();
            solveBtn.disabled = true;
            solveBtn.textContent = 'Solving...';
            setPlaying(false);

            const formData = new URLSearchParams();
            formData.set('boardText', boardText.value);
            formData.set('algorithm', algorithm.value);
            formData.set('heuristic', heuristic.value);
            formData.set('weighted',
document.getElementById('weighted').value || '1.8');

            try {
                const response = await fetch('/solve', { method: 'POST',
body: formData });
                const payload = await response.json();
                if (!response.ok || !payload.ok) {
                    throw new Error(payload.error || 'Solve request
failed');
                }

                frames = payload.frames || [];
                labels = payload.frameMoves || [];
                previousActorIndex = -1;
                statFound.textContent = payload.found ? 'Yes' : 'No';
                statFound.className = 'v ' + (payload.found ? 'ok' :
'warn');

                statCost.textContent = String(payload.totalCost ?? '-');
                statExpanded.textContent = String(payload.expanded ??
'-');

                statElapsed.textContent = Number(payload.elapsedMS ||
0).toFixed(3) + ' ms';

                downloadBtn.classList.remove('hidden');

                if (!frames.length) {
                    frames = [['No state available']];
                    labels = ['Initial'];
                }

                frameSlider.disabled = frames.length <= 1;
                prevBtn.disabled = frames.length <= 1;
                nextBtn.disabled = frames.length <= 1;
                playBtn.disabled = frames.length <= 1;
                frameSlider.min = '0';
                frameSlider.max = String(Math.max(0, frames.length - 1));
            }

```



```

        renderFrame(0);
        // Auto-start playback when the solver finishes and there
is more than one frame
        if (frames.length > 1) {
            setPlaying(true);
        }
    } catch (err) {
        showError(err.message || String(err));
    } finally {
        solveBtn.disabled = false;
        solveBtn.textContent = 'Run solver';
    }
}

solveBtn.addEventListener('click', runSolver);
solveForm.addEventListener('submit', (event) => {
    event.preventDefault();
    runSolver();
});

    renderFrame(0);
</script>
</body>
</html>

```

3.7 main.go

```

package main

import (
    "bufio"
    "flag"
    "fmt"
    "os"
    "strings"
    "time"
)

func promptLine(prompt string) string{
    fmt.Print(prompt)
    rd := bufio.NewReader(os.Stdin)
    s, _ := rd.ReadString('\n')
    return strings.TrimSpace(s)
}

func saveSolution(path string, b *Board, res SolveResult, elapsed float64) error{
    f, err := os.Create(path)
    if err!=nil { return err }
    defer f.Close()
    w := bufio.NewWriter(f)
    if res.Found{
        fmt.Fprintf(w, "Solusi: %s\n", res.Moves)
        fmt.Fprintf(w, "Cost: %d\n", res.TotalCost)
        fmt.Fprintf(w, "Iterasi: %d\n", res.Expanded)
        fmt.Fprintf(w, "Waktu(ms): %.3f\n\n", elapsed)
        for i, st := range res.StatePath{
            if i==0 { fmt.Fprintln(w, "Initial") } else { fmt.Fprintf(w, "Step %d:

```

```

%s\n", i, string(res.Moves[i-1])) }
        for _, line := range RenderBoard(b, st.Pos, st.NextDigit){
fmt.Fprintln(w, line) }
        fmt.Fprintln(w, "")
    }
    } else {
fmt.Fprintln(w, "Tidak ada solusi valid.")
    }
    w.Flush()
    return nil
}

func runCLI(){
    fmt.Println("Ice Sliding Puzzle Solver (Go)")
    filePath := promptLine(">> Masukkan file input (.txt): ")
    if filePath=="" { fmt.Println("Tidak ada input."); return }
    board, err := LoadBoard(filePath)

    if err!=nil { fmt.Println("Input tidak valid:", err); return }

    algorithm := strings.ToUpper(promptLine(">> Pilih algoritma
(UCS/GBFS/A*/WASTAR): "))
    heuristic := "H1"
    if algorithm=="GBFS" || algorithm=="A*" || algorithm=="WASTAR"{
        heuristic = strings.ToUpper(promptLine(">> Pilih heuristic (H1/H2/H3/H4/H5):
"))
    }
    start := time.Now()
    res, err := Solve(board, algorithm, heuristic, 1.8)
    if err!=nil { fmt.Println("Konfigurasi tidak valid:", err); return }
    elapsed := float64(time.Since(start).Milliseconds())

    if res.Found{
        fmt.Println("\nSolusi ditemukan")
        fmt.Println("Gerakan:", res.Moves)
        fmt.Println("Cost solusi:", res.TotalCost)
    } else { fmt.Println("\nTidak ada solusi valid.") }

    fmt.Printf("\nWaktu eksekusi: %.3f ms\n", elapsed)
    fmt.Printf("Banyak iterasi ditinjau: %d\n", res.Expanded)

    pb := strings.ToLower(promptLine(">> Jalankan playback? (Ya/Tidak): "))
    if (pb=="ya"||pb=="y"||pb=="yes") && res.Found{
        for i, st := range res.StatePath{
            fmt.Printf("\nStep %d\n", i)
            for _, l := range RenderBoard(board, st.Pos, st.NextDigit){
fmt.Println(l) }
                if i < len(res.StatePath)-1 { _ = promptLine("Tekan Enter untuk langkah
berikutnya...") }
            }
        }

        save := strings.ToLower(promptLine(">> Simpan solusi ke file? (Ya/Tidak): "))
        if save=="ya"||save=="y"||save=="yes"{
            out := promptLine(">> Lokasi file output (mis. test/solusi.txt): ")
            if out=="" { out = "test/solusi.txt" }
            if err := saveSolution(out, board, res, elapsed); err!=nil {
fmt.Println("Gagal menyimpan file:", err) } else { fmt.Println("Solusi disimpan
ke:", out) }
        }
    }
}

```

```

func main(){
    cliMode := flag.Bool("cli", false, "launch CLI")
    flag.Parse()
    if *cliMode {
        runCLI()
        return
    }
    runGUI()
}

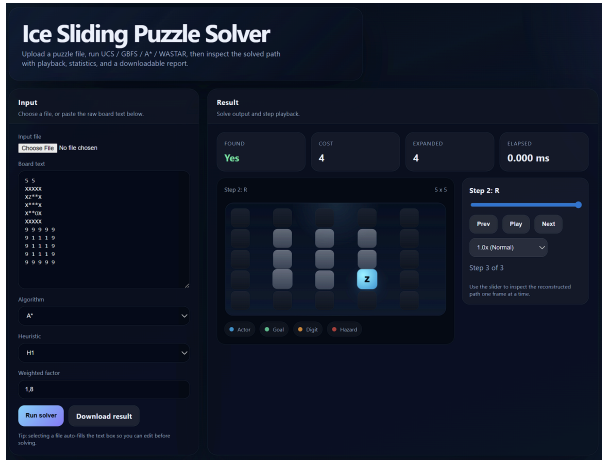
```

4. Tangkapan Layar Input dan Output

Berikut adalah tangkapan layar yang memperlihatkan input dan output program pada berbagai test case:

4.1 Test Case 1: Pergerakan Dasar (Tanpa Angka)

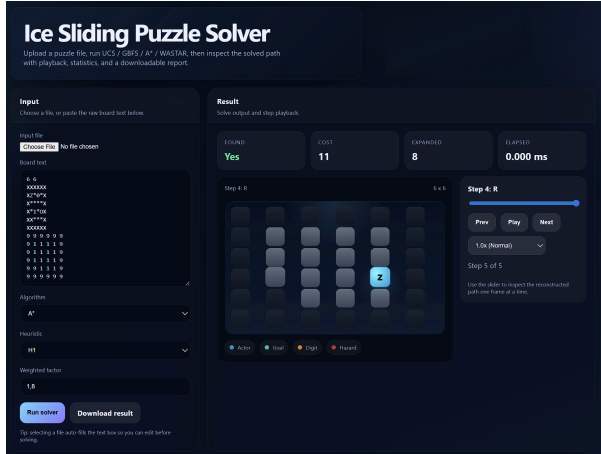
Deskripsi: Papan berukuran 5x5 sederhana untuk menguji apakah aktor (Z) dapat meluncur dan berhenti di batu (X) untuk mencapai tujuan (O). Algoritma yang digunakan adalah UCS.

Input	Output
<pre> 5 5 XXXXX XZ**X X***X X**OX XXXXX 9 9 9 9 9 9 1 1 1 9 9 1 1 1 9 9 1 1 1 9 9 9 9 9 9 </pre>	 The screenshot shows the 'Ice Sliding Puzzle Solver' interface. On the left, the 'Input' section displays a 5x5 grid with the puzzle state: <pre> 5 5 XXXXX XZ**X X***X X**OX XXXXX 9 9 9 9 9 9 1 1 1 9 9 1 1 1 9 9 1 1 1 9 9 9 9 9 9 </pre> The 'Algorithm' is set to 'A*' and the 'Heuristic' is 'H1'. The 'Result' section on the right shows the solution: 'Solved: Yes', 'Cost: 4', 'Expanded: 4', and 'Time: 0.000 ms'. It also displays a 5x5 grid with the solution path, where the actor 'Z' is at the bottom center and the goal 'O' is at the top center.

4.2 Test Case 2: Constraint Pengambilan Angka Berurutan

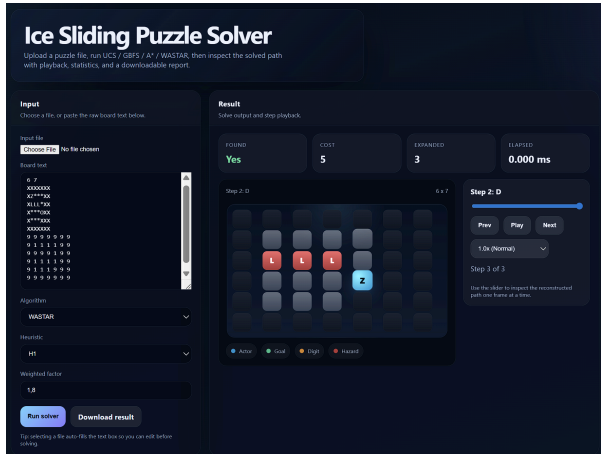
Deskripsi: Papan berukuran 6x6 di mana aktor harus mengambil angka 0 terlebih dahulu, kemudian angka 1, sebelum akhirnya boleh menuju titik tujuan (O). Algoritma yang digunakan adalah A* dengan heuristik H1.

Input	Output
-------	--------

<pre> 6 6 XXXXXX XZ*0*X X****X X*1*OX XX***X XXXXXX 999999 911119 911119 911119 991119 999999 </pre>	
--	--

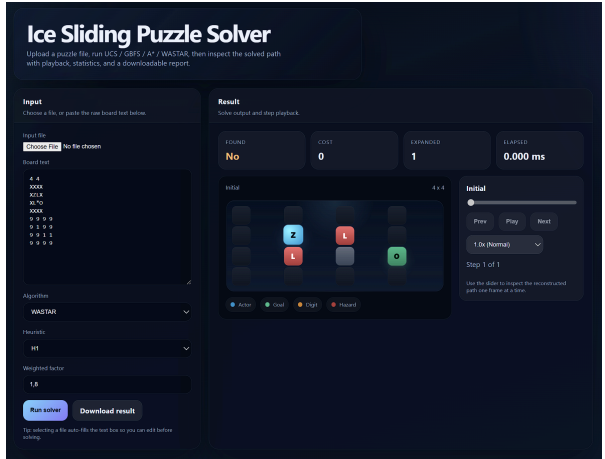
4.3 Test Case 3: Menghindari Halangan Lava

Deskripsi: Papan 6x7 yang mendemonstrasikan bahwa algoritma tidak akan memilih jalur yang melewati Lava (L) karena akan menyebabkan game over. Aktor harus memutar mencari batu sandaran lain. Algoritma yang digunakan adalah WA*.

Input	Output
<pre> 6 7 XXXXXXX XZ***XX XLLL*XX X***OXX X***XXX XXXXXXX 9999999 9111199 9999199 9111199 9111999 9999999 </pre>	

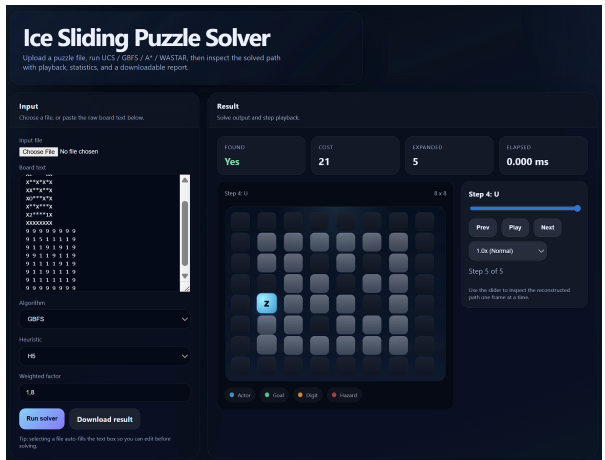
4.4 Test Case 4: Tidak Ada Solusi (Jalan Buntu)

Deskripsi: Kasus uji negatif di mana aktor (Z) terkurung oleh rintangan batu (X) dan Lava (L), sehingga sama sekali tidak ada jalan menuju titik tujuan (O). Program harus bisa mendeteksi bahwa solusi tidak ditemukan.

Input	Output
<pre> 4 4 XXXX XZLX XL*O XXXX 9 9 9 9 9 1 9 9 9 9 1 1 9 9 9 9 </pre>	

4.5 Test Case 5: Papan Skala Besar (Variasi Cost)

Deskripsi: Papan berukuran 8x8 yang lebih kompleks dengan variasi cost traversal (ada petak dengan cost tinggi dan rendah). Pengujian ini membandingkan kemampuan heuristik dalam mencari jalan termurah. Algoritma yang digunakan adalah GBFS dengan heuristik H5.

Input	Output
<pre> 8 8 XXXXXXXXXX XZ****0X X**X*X*X XX**X**X XO***X*X X**X***X X2****1X XXXXXXXXXX 9 9 9 9 9 9 9 9 9 1 5 1 1 1 1 9 9 1 1 9 1 9 1 9 9 9 1 1 9 1 1 9 9 1 1 1 1 9 1 9 9 1 1 9 1 1 1 9 9 1 1 1 1 1 1 9 9 9 9 9 9 9 9 9 </pre>	

5. Analisis Hasil Percobaan

Untuk menganalisis performa algoritma pathfinding yang diimplementasikan, kami melakukan uji coba menggunakan satu test case papan yang sama dengan berbagai konfigurasi algoritma dan heuristik. Berikut adalah perbandingan hasilnya:

Algoritma	Heuristik	Total Cost (Biaya)	Expanded Nodes (Iterasi)	Waktu Eksekusi (ms)
UCS	-	21	13	< 1 ms
GBFS	H1 (Manhattan)	21	5	< 1 ms
A*	H1 (Manhattan)	21	9	< 1 ms
A*	H5 (Digit Penalty)	21	9	< 1 ms
WA* ($w=1.8$)	H1 (Manhattan)	21	9	< 1 ms

Berdasarkan hasil percobaan di atas, diperoleh analisis sebagai berikut:

1. Uniform Cost Search (UCS)

UCS menjamin solusi yang didapatkan sangat optimal, dibuktikan dengan perolehan Total Cost sebesar 21. Namun, karena algoritma ini menelusuri (expand) state ke segala arah secara buta karena tidak dituntun oleh heuristik (hanya mengandalkan $g(n)$), UCS menghasilkan jumlah iterasi yang paling banyak (13 expanded nodes) dibandingkan algoritma lainnya.

2. Greedy Best-First Search (GBFS)

GBFS meninjau jumlah node yang paling sedikit (hanya 5 iterasi). Hal ini karena GBFS bergerak secara agresif ke arah target berdasarkan nilai heuristik saja ($f(n) = h(n)$). Meskipun GBFS pada teori umumnya rentan terjebak pada jalur yang memutar sehingga tidak menjamin solusi optimal, pada test case khusus ini GBFS kebetulan berhasil menemukan rute dengan Total Cost yang optimal (21) dengan sangat efisien.

3. A* (A-Star)

Algoritma A* menawarkan keseimbangan yang baik antara efisiensi iterasi dan optimalitas solusi. Karena heuristik H1 yang digunakan bersifat admissible, A* berhasil menemukan Total Cost yang optimal (21) sama dengan hasil UCS. Namun berkat panduan heuristik, penelusuran A* lebih terarah sehingga jumlah iterasinya (9 node) lebih sedikit dibandingkan UCS, membuatnya lebih mangkus secara memori.

4. Weighted A* (WA*)

Pemberian bobot ($\text{weight} = 1.8$) pada nilai heuristik secara teoritis membuat A* bersikap lebih greedy. Namun, dari hasil percobaan ini, WA* menghasilkan Total Cost dan jumlah iterasi yang sama persis dengan A* biasa (9 iterasi). Hal ini terjadi karena skala test case map yang digunakan relatif kecil, sehingga pembobotan heuristik tidak memberikan perbedaan rute eksplorasi yang signifikan dibandingkan A* standar. Pada map yang jauh lebih masif, WA* akan menunjukkan percepatan iterasi yang lebih nyata.

5. Pengaruh Pemilihan Heuristik

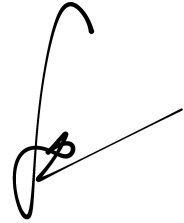
Pada pengujian algoritma A*, penggunaan heuristik kompleks H5 (yang memperhitungkan penalti sisa digit) menghasilkan performa iterasi yang sama dengan heuristik H1 yang lebih naif (sama-sama 9 iterasi). Hal ini dikarenakan letak antar digit pada test case uji coba tidak terlalu rumit. Akibatnya, keunggulan H5 dalam memfokuskan pencarian jalur berdasarkan urutan sisa digit belum terlihat pemangkasannya secara signifikan pada skala peta ini.

Catatan Tambahan Terkait Waktu Eksekusi:

Seluruh algoritma mencatatkan waktu eksekusi yang sangat instan, yaitu di bawah 1 milidetik ($< 1 \text{ ms}$). Hal ini wajar mengingat ukuran test case yang tidak terlalu besar dipadukan dengan performa kompilasi bahasa pemrograman Go yang sangat cepat dalam mengeksekusi logika iterasi. Oleh karena itu, metrik utama perbandingan performa difokuskan pada efisiensi jumlah Expanded Nodes.

6. Surat Pernyataan

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (Generative AI), melainkan hasil pemikiran dan analisis mandiri.



Leonard Arif Sutiono



Muhammad Azzam Robbani

LAMPIRAN

No	Poin	Ya	Tidak
1	Program berhasil di kompilasi tanpa kesalahan	✓	
2	Program berhasil dijalankan	✓	
3	Solusi yang diberikan program benar dan mematuhi aturan permainan	✓	
4	Program dapat membaca masukan berkas .txt serta menyimpan solusi dalam berkas .txt	✓	
5	Program memiliki Graphical User Interface (GUI)	✓	

6	Ada algoritma pathfinding alternatif selain dari spesifikasi wajib (Algoritma X dan Algoritma Y)	✓	
7	Ada tambahkan dua alternatif heuristik selain yang utama (Heuristik X dan Heuristik Y)	✓	

Link Repository:

https://github.com/LeonArif/Tucil3_18223025_18223120